05. SP1 Core Machine 분석

개요

SP1 Core Machine은 RISC-V 실행 추적을 산술화(arithmetization)하여 영지식 증명을 생성하는 핵심 시스템입니다. 이 컴포넌트는 AIR(Algebraic Intermediate Representation) 시스템을 통해 실행 추적을 다항식 제약 조건으로 변환하고, 다양한 특화된 칩(Chip)들을 통해 효율적인 증명 생성을 가능하게 합니다.

AIR (Algebraic Intermediate Representation) 시스템

1. AIR 개념 및 구조

AIR는 계산을 다항식 제약 조건의 집합으로 표현하는 방법입니다. SP1에서는 각 RISC-V 명령어와 연산을 별도의 AIR로 구현하여 모듈화된 증명 시스템을 구축합니다.

```
/// SP1의 RISC-V 실행을 인코딩하는 AIR

pub  enum RiscvAir<F: PrimeField32> {

/// 프로그램 테이블과 명령어 룩업을 포함하는 AIR

Program(ProgramChip),

/// RISC-V CPU AIR (각 행은 CPU 사이클을 나타냄)

Cpu(CpuChip),

/// RISC-V Add/SUB 명령어 AIR

Add(AddSubChip),

/// RISC-V 비트 연산 명령어 AIR
```

```
Bitwise(BitwiseChip),

/// RISC-V 곱셈 명령어 AIR

Mul(MulChip),

/// RISC-V 나눗셈/나머지 명령어 AIR

DivRem(DivRemChip),

/// 메모리 명령어 AIR

Memory(MemoryInstructionsChip),

/// 시스템 콜 AIR

SyscallCore(SyscallChip),

/// 암호화 프리컴파일 AIR들

Sha256Extend(ShaExtendChip),

Sha256Compress(ShaCompressChip),

Ed25519Add(EdAddAssignChip<EdwardsCurve<Ed25519Parameters>>),

// ... 기타 다양한 AIR들

}
```

2. AIR 빌더 시스템

```
/// SP1 Core AIR 빌더 트레이트

pub  trait SP1CoreAirBuilder:

SP1AirBuilder + WordAirBuilder + MemoryAirBuilder + ProgramAirBuilder

{

}
```

```
/// 메모리 상호작용을 위한 AIR 빌더
pub trait MemoryAirBuilder {
/// 메모리 접근 검증
fn verify_mem_access_ts(&mut self, addr: Self::Var, timestamp: Self::Var);
/// 메모리 읽기 전송
fn send_memory_read(&mut self, addr: Self::Var, value: Self::Var, timestamp: S
/// 메모리 쓰기 전송
fn send_memory_write(&mut self, addr: Self::Var, value: Self::Var, timestamp:
}


/// 프로그램 상호작용을 위한 AIR 빌더
pub trait ProgramAirBuilder {
/// 프로그램 명령어 전송
fn send_program(&mut self, pc: Self::Var, instruction: Self::Var, is_real: Sel
}
```

칩(Chip) 아키텍처

1. CPU 칩 - 중앙 조정자

CPU 칩은 모든 명령어 실행의 중심 역할을 하며, 다른 특화된 칩들과의 상호작용을 조정합니다.

```
impl<AB> Air<AB> for CpuChip
```

```
where
AB: SP1CoreAirBuilder + AirBuilderWithPublicValues,
{
fn  eval(&self, builder: &mut AB) {
let  main = builder.main();
let (local, next) = (main.row_slice(0), main.row_slice(1));
let  local: &CpuCols<AB::Var> = (*local).borrow();
let  next: &CpuCols<AB::Var> = (*next).borrow();

// 클럭 제약 조건 (16비트 + 8비트 리밍)
let  clk = AB::Expr::from_canonical_u32(1u32 << 16) * local.clk_8bit_limb
+ local.clk_16bit_limb;

// 프로그램 제약 조건
builder.send_program(local.pc, local.instruction, local.is_real);

// 레지스터 제약 조건
self.eval_registers::<AB>(builder, local, clk.clone());

// 명령어 전송 (다른 칩들이 수신)
builder.send_instruction(
local.shard_to_send,
local.clk_to_send,
```

```
local.pc,
local.next_pc,
local.num_extra_cycles,
local.instruction.opcode,
local.op_a_val(),
local.op_b_val(),
local.op_c_val(),
local.instruction.op_a_0,
local.op_a_immutable,
local.is_memory,
local.is_syscall,
local.is_halt,
local.is_real,
);

// 샤드 및 클럭 업데이트 검증
self.eval_shard_clk(builder, local, next, public_values, clk.clone());
// PC 업데이트 검증
self.eval_pc(builder, local, next, public_values);
}
}
```

CPU 칩의 주요 제약 조건:

- **클럭 관리**: 24비트 클럭을 16비트 + 8비트로 분해하여 범위 검사
- **PC 진행**: 기본적으로 4씩 증가, 분기/점프 시 예외 처리
- **레지스터 관리**: 32개 RISC-V 레지스터 상태 추적
- **명령어 디스패치**: 적절한 특화 칩으로 명령어 전달

2. ALU 칩들 - 산술 연산 특화

AddSub 칩 예시

```
#[derive(AlignedBorrow, Default, Clone, Copy)]
#[repr(C)]
pub struct AddSubCols<T> {
/// 프로그램 카운터
pub pc: T,
/// 덧셈 연산 인스턴스
pub add_operation: AddOperation<T>,
/// 첫 번째 피연산자
pub operand_1: Word<T>,
/// 두 번째 피연산자
pub operand_2: Word<T>,
/// 첫 번째 피연산자가 레지스터 0이 아닌지 여부
pub op_a_not_0: T,
/// ADD 연산 여부
pub is_add: T,
/// SUB 연산 여부
```

```
pub  is_sub: T,
}

impl<AB> Air<AB> for AddSubChip
where
AB: SP1AirBuilder,
{
fn  eval(&self, builder: &mut AB) {
let  main = builder.main();
let  local = main.row_slice(0);
let  local: &AddSubCols<AB::Var> = (*local).borrow();

// 선택자 제약 조건
let  is_real = local.is_add + local.is_sub;
builder.assert_bool(local.is_add);
builder.assert_bool(local.is_sub);
builder.assert_bool(is_real.clone());

// 연산 코드 계산
let  opcode = AB::Expr::from_f(Opcode::ADD.as_field()) * local.is_add +
AB::Expr::from_f(Opcode::SUB.as_field()) * local.is_sub;

// 덧셈 연산 평가
```

```
AddOperation::<AB::F>::eval(
builder,
local.operand_1,
local.operand_2,
local.add_operation,
local.op_a_not_0.into(),
);


// CPU로부터 명령어 수신
builder.receive_instruction(
AB::Expr::zero(),
AB::Expr::zero(),
local.pc,
local.pc + AB::Expr::from_canonical_u32(DEFAULT_PC_INC),
AB::Expr::zero(),
opcode,
local.add_operation.value,
local.operand_1,
local.operand_2,
AB::Expr::one() - local.op_a_not_0,
AB::Expr::zero(),
AB::Expr::zero(),
AB::Expr::zero(),
```

```
AB::Expr::zero(),
local.is_add,
);
}
}
```

3. 메모리 칩들 - 메모리 일관성 보장

글로벌 메모리 칩

```
pub struct MemoryGlobalChip {
pub kind: MemoryChipType,
}
```

```
#[derive(AlignedBorrow, Clone, Copy)]
#[repr(C)]
pub struct MemoryInitCols<T: Copy> {
/// 샤드 번호
pub shard: T,
/// 타임스탬프
pub timestamp: T,
/// 메모리 주소
pub addr: T,
/// 주소 순서 비교를 위한 제약 조건
```

```rust
pub lt_cols: AssertLtColsBits<T, 32>,
/// 주소의 비트 분해
pub addr_bits: BabyBearBitDecomposition<T>,
/// 메모리 값 (32비트)
pub value: [T; 32],
/// 실제 접근 여부
pub is_real: T,
/// 다음 주소와 비교 여부
pub is_next_comp: T,
/// 이전 주소가 0인지 확인하는 연산
pub is_prev_addr_zero: IsZeroOperation<T>,
/// 첫 번째 비교 여부
pub is_first_comp: T,
/// 마지막 주소 플래그
pub is_last_addr: T,
}
```

메모리 칩의 핵심 제약 조건:

- **주소 순서**: 메모리 접근이 주소 순으로 정렬되어 있음을 보장
- **값 일관성**: 초기화와 최종화 시점의 값 일치성 검증
- **범위 검사**: 주소와 값이 유효한 범위 내에 있음을 확인

제약 조건 생성 과정

1. 다항식 제약 조건의 구조

SP1의 각 칩은 다음과 같은 형태의 다항식 제약 조건을 생성합니다:

```
// 기본 제약 조건 형태
P(x) = Σ(αᵢ * Cᵢ(x)) = 0

// 여기서:
// - P(x): 전체 제약 조건 다항식
// - αᵢ: 랜덤 선형 결합 계수
// - Cᵢ(x): 개별 제약 조건
```

2. 제약 조건 평가 과정

```
impl<AB> Air<AB> for AddSubChip
where
AB: SP1AirBuilder,
{
fn  eval(&self, builder: &mut AB) {
// 1. 로컬 변수 추출
let  local: &AddSubCols<AB::Var> = (*local).borrow();
// 2. 불린 제약 조건
builder.assert_bool(local.is_add);
builder.assert_bool(local.is_sub);
```

```
// 3. 산술 제약 조건
let  result = local.operand_1 + local.operand_2;
builder.assert_eq(local.add_operation.value, result);
// 4. 조건부 제약 조건
builder.when(local.is_add).assert_eq(local.result, result);
// 5. 상호작용 제약 조건
builder.receive_instruction(/* ... */);
}
}
```

3. 제약 조건 유형

기본 제약 조건

```
// 불린 제약 조건
builder.assert_bool(x); // x * (1 - x) = 0

// 등식 제약 조건
builder.assert_eq(a, b); // a - b = 0

// 0 제약 조건
builder.assert_zero(x); // x = 0
```

조건부 제약 조건

```
// 조건부 등식
builder.when(condition).assert_eq(a, b); // condition * (a - b) = 0


// 조건부 0 제약
builder.when(condition).assert_zero(x); // condition * x = 0
```

범위 제약 조건

```
// 바이트 범위 검사
builder.send_byte(
AB::Expr::from_canonical_u8(ByteOpcode::U8Range as u8),
value,
AB::Expr::zero(),
AB::Expr::zero(),
is_real,
);


// 24비트 범위 검사
builder.eval_range_check_24bits(
value,
low_16_bits,
high_8_bits,
is_real,
```

```
);
```

상호작용 시스템

1. 룩업 아규먼트

SP1은 다양한 칩 간의 일관성을 보장하기 위해 룩업 아규먼트를 사용합니다.

```
/// 상호작용 종류
pub enum InteractionKind {
Memory, // 메모리 접근
Program, // 프로그램 명령어
Syscall, // 시스템 콜
Byte, // 바이트 연산
Global, // 글로벌 상태
}

/// 상호작용 범위
pub enum InteractionScope {
Local, // 샤드 내부
Global, // 샤드 간
}
```

2. 메모리 일관성 검증

```
impl MemoryAirBuilder for AB {
fn verify_mem_access_ts(&mut self, addr: Self::Var, timestamp: Self::Var) {
// 타임스탬프 = 샤드 * 2^24 + 클럭
let computed_ts = shard * AB::F::from_canonical_u32(1 << 24) + clk;
self.assert_eq(timestamp, computed_ts);
}
fn send_memory_read(&mut self, addr: Self::Var, value: Self::Var, timestamp: S
self.send(AirInteraction::new(
vec![addr, value, timestamp, AB::Expr::zero()], // is_write = 0
is_real,
InteractionKind::Memory,
));
}
}
```

3. 프로그램 일관성 검증

```
impl ProgramAirBuilder for AB {
fn send_program(&mut self, pc: Self::Var, instruction: Self::Var, is_real: Sel
self.send(AirInteraction::new(
vec![pc, instruction],
is_real,
```

```
InteractionKind::Program,
));
}
}
```

산술화 과정

1. 실행 추적에서 다항식으로

```
impl<F: PrimeField32> MachineAir<F> for AddSubChip {
fn generate_trace(&self, input: &ExecutionRecord, _: &mut ExecutionRecord) -> R
let merged_events = input.add_events.iter().chain(input.sub_events.iter());
let padded_nb_rows = self.num_rows(input).unwrap();
let mut values = zeroed_f_vec(padded_nb_rows * NUM_ADD_SUB_COLS);

// 이벤트를 행렬 행으로 변환
for (idx, event) in merged_events.enumerate() {
let row_start = idx * NUM_ADD_SUB_COLS;
let row = &mut values[row_start..row_start + NUM_ADD_SUB_COLS];
let cols: &mut AddSubCols<F> = row.borrow_mut();
// 이벤트 데이터를 칼럼에 채움
cols.pc = F::from_canonical_u32(event.pc);
cols.is_add = F::from_bool(event.opcode == Opcode::ADD);
```

```
cols.is_sub = F::from_bool(event.opcode == Opcode::SUB);

// 연산 결과 계산 및 저장

let operand_1 = if event.opcode == Opcode::ADD { event.b } else { event.a };

cols.add_operation.populate(&mut byte_lookup_events, operand_1, event.c);

cols.operand_1 = Word::from(operand_1);

cols.operand_2 = Word::from(event.c);

}


RowMajorMatrix::new(values, NUM_ADD_SUB_COLS)

}

}
```

2. 다항식 커밋 과정

```
// 1. 추적 행렬 생성

let trace_matrix = chip.generate_trace(&execution_record);


// 2. 다항식 보간

let trace_polys = trace_matrix.interpolate();


// 3. 제약 조건 평가

let constraint_polys = chip.eval_constraints(&trace_polys);
```

```
// 4. 다항식 커밋
let commitments = commit_polynomials(&constraint_polys);
```

최적화 기법

1. 칩 특화 최적화

```
impl AddSubChip {
/// 이벤트를 행으로 변환 (최적화된 버전)
fn event_to_row_optimized<F: PrimeField>(
&self,
event: &AluEvent,
cols: &mut AddSubCols<F>,
blu: &mut impl ByteRecord,
) {
// 브랜치 예측 최적화
let is_add = event.opcode == Opcode::ADD;
cols.is_add = F::from_bool(is_add);
cols.is_sub = F::from_bool(!is_add);

// 조건부 연산 최적화
let (operand_1, operand_2) = if is_add {
(event.b, event.c)
```

```
} else {
(event.a, event.c)
};

// 인라인 연산으로 성능 향상
cols.add_operation.populate_inline(blu, operand_1, operand_2);
}
}
```

2. 메모리 접근 최적화

```
impl MemoryGlobalChip {
fn  generate_trace_optimized(&self, input: &ExecutionRecord) -> RowMajorMatrix<F
let  mut  memory_events = match  self.kind {
MemoryChipType::Initialize => input.global_memory_initialize_events.clone(),
MemoryChipType::Finalize => input.global_memory_finalize_events.clone(),
};

// 주소 기준 정렬 (메모리 지역성 활용)
memory_events.sort_by_key(|event| event.addr);

// 병렬 처리를 통한 성능 향상
let  rows: Vec<[F; NUM_MEMORY_INIT_COLS]> = memory_events
```

```
.par_iter()
.map(|event| {
let mut row = [F::zero(); NUM_MEMORY_INIT_COLS];
self.populate_row_optimized(event, &mut row);
row
})
.collect();

RowMajorMatrix::new(rows.into_iter().flatten().collect(), NUM_MEMORY_INIT_COLS)
}
}
```

3. 제약 조건 최적화

```
impl<AB> Air<AB> for OptimizedChip
where
AB: SP1AirBuilder,
{
fn eval(&self, builder: &mut AB) {
// 제약 조건 그룹화로 평가 횟수 감소
let common_condition = local.is_real * local.is_valid;
builder.when(common_condition.clone()).assert_eq(a, b);
builder.when(common_condition.clone()).assert_eq(c, d);
```

```
builder.when(common_condition).assert_eq(e, f);

// 미리 계산된 값 재사용

let precomputed_sum = local.operand_1 + local.operand_2;

builder.assert_eq(local.result, precomputed_sum);

}

}
```

디버깅 및 검증

1. 제약 조건 검증

```
#[cfg(test)]

mod tests {

use super::*;

#[test]

fn test_constraint_satisfaction() {

let chip = AddSubChip::default();

let mut shard = ExecutionRecord::default();

// 테스트 이벤트 생성

shard.add_events.push(AluEvent::new(0, Opcode::ADD, 5, 2, 3, false));

// 추적 생성

let trace = chip.generate_trace(&shard, &mut ExecutionRecord::default());

// 제약 조건 검증
```

```
let  config = BabyBearPoseidon2::new();

let  mut  challenger = config.challenger();

let  proof = prove(&config, &chip, &mut  challenger, trace);

// 검증

let  mut  challenger = config.challenger();

verify(&config, &chip, &mut  challenger, &proof).unwrap();

}

}
```

2. 상호작용 디버깅

```
#[test]

fn  test_memory_interactions() {

let  machine = RiscvAir::machine(BabyBearPoseidon2::new());

let (pkey, _) = machine.setup(&program);

// 메모리 상호작용 디버깅

debug_interactions_with_all_chips(

&machine,

&pkey,

&shards,

vec![InteractionKind::Memory],

InteractionScope::Local,

);
```

}

요약

SP1 Core Machine은 정교한 AIR 시스템과 특화된 칩 아키텍처를 통해 RISC-V 실행을 효율적으로 산술화합니다. 각 칩은 특정 연산에 최적화되어 있으며, 상호작용 시스템을 통해 전체 시스템의 일관성을 보장합니다. 이러한 모듈화된 설계는 높은 성능과 확장성을 제공하며, SP1이 복잡한 프로그램에 대해서도 효율적인 영지식 증명을 생성할 수 있게 합니다.